

Name: M.Rishitha
Course Code: DSA0306
processing

RegisterNo:192424335
Course Name:Natural Language

Experiment-1:Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Code:

Program 1: Regular Expressions in Python

```
import re

text = "My phone number is 9876543210 and my email is student123@gmail.com"

# Search for a phone number
phone = re.search(r"\d{10}", text)

# Search for email
email = re.search(r"\S+@\S+", text)

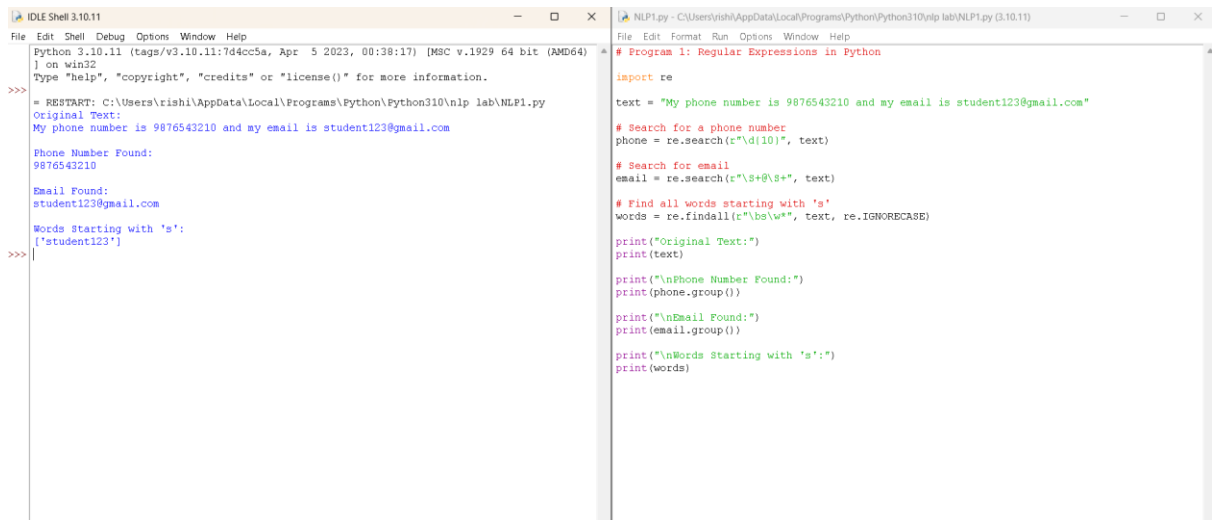
# Find all words starting with 's'
words = re.findall(r"\bs\w*", text, re.IGNORECASE)

print("Original Text:")
print(text)

print("\nPhone Number Found:")
print(phone.group())

print("\nEmail Found:")
print(email.group())

print("\nWords Starting with 's':")
print(words)
```



```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\trishi\AppData\Local\Programs\Python\Python310\nlp lab\NLP1.py
Original Text:
My phone number is 9876543210 and my email is student123@gmail.com

Phone Number Found:
9876543210

Email Found:
student123@gmail.com

Words Starting with 's':
['student123']
>>>
```

```
# Program 1: Regular Expressions in Python
import re

text = "My phone number is 9876543210 and my email is student123@gmail.com"

# Search for a phone number
phone = re.search(r"\d{10}", text)

# Search for email
email = re.search(r"\S+@\S+", text)

# Find all words starting with 's'
words = re.findall(r"[a-zA-Z]\b", text, re.IGNORECASE)

print("Original Text:")
print(text)

print("\nPhone Number Found:")
print(phone.group())

print("\nEmail Found:")
print(email.group())

print("\nWords Starting with 's':")
print(words)
```

Experiment 2: Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Python Code:

Program 2: Finite State Automaton

Recognizes strings ending with 'ab'

def fsa(string):

 state = "q0"

 for char in string:

 if state == "q0":

 if char == "a":

 state = "q1"

 else:

 state = "q0"

 elif state == "q1":

 if char == "b":

 state = "q2"

 elif char == "a":

 state = "q1"

 else:

 state = "q0"

```

elif state == "q2":
    if char == "a":
        state = "q1"
    else:
        state = "q0"
if state == "q2":
    return True
else:
    return False

strings = ["ab", "aab", "aaab", "abb", "baba", "xyzab"]
for s in strings:
    print(s, "->", fsa(s))

```

The screenshot shows two windows from a Python IDE. The left window is a terminal showing the execution of the `fsa` function for the strings `ab`, `aab`, `aaab`, `abb`, `baba`, and `xyzab`. The output shows that `ab`, `aab`, and `aaab` return `True`, while `abb`, `baba`, and `xyzab` return `False`. The right window shows the source code of the `fsa` function, which implements a finite state automaton to recognize strings ending with 'ab'.

```

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:\Users\trishi\AppData\Local\Programs\Python\Python310\nlp lab\nlp2.py
ab -> True
aab -> True
aaab -> True
abb -> False
baba -> False
xyzab -> True
>>>

```

```

# Program 2: Finite State Automaton
# Recognizes strings ending with 'ab'

def fsa(string):
    state = "q0"
    for char in string:
        if state == "q0":
            if char == "a":
                state = "q1"
            else:
                state = "q0"
        elif state == "q1":
            if char == "b":
                state = "q2"
            elif char == "a":
                state = "q1"
            else:
                state = "q0"
        elif state == "q2":
            if char == "a":
                state = "q1"
            else:
                state = "q0"
    if state == "q2":
        return True
    else:
        return False

strings = ["ab", "aab", "aaab", "abb", "baba", "xyzab"]
for s in strings:
    print(s, "->", fsa(s))

```

Experiment 3

1. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Python code:

```

import nltk

from nltk.stem import WordNetLemmatizer

from nltk.tokenize import word_tokenize

from nltk.corpus import wordnet

# Download required resources (Run only once)

```

```

nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('averaged_perceptron_tagger')

# Create lemmatizer object
lemmatizer = WordNetLemmatizer()

# Input sentence
text = "The boys are running, the girls are writing letters, and the children played
games."

# Tokenize the sentence
words = word_tokenize(text)

# Perform POS tagging
pos_tags = nltk.pos_tag(words)

# Function to convert NLTK POS tags to WordNet POS
def get_wordnet_pos(tag):
    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB
    elif tag.startswith('N'):
        return wordnet.NOUN
    elif tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

print("Word\t\tPOS\tLemma")
print("-" * 35)

# Perform Morphological Analysis
for word, tag in pos_tags:
    lemma = lemmatizer.lemmatize(word, get_wordnet_pos(tag))
    print(f'{word:12} {tag:8} {lemma}')

```

The image shows two side-by-side IDE windows. The left window, titled 'IDLE Shell 3.10.11', displays the output of running 'nltk.download()' for various packages. It shows progress bars and status messages for 'punkt', 'wordnet', 'omw-1.4', and 'averaged_perceptron_tagger'. Below the output, a table displays the POS and Lemma for the words in the sentence 'The boys are running, the girls are writing letters, and the children played games.'.

Word	POS	Lemma
The	DT	The
boys	NNS	boy
are	VBP	be
running	VBG	run
,	,	,
the	DT	the
girls	NNS	girl
are	VBP	be
writing	VBG	write
letters	NNS	letter
,	,	,
and	CC	and
the	DT	the
children	NNS	child
played	VED	play
games	NNS	game
.	.	.

The right window, titled 'nlp3.py - C:\Users\rishi\AppData\Local\Programs\Python\Python310\lab\nlp3.py (3.10.11)', shows the Python code used for the processing. It imports NLTK modules, downloads resources, creates a lemmatizer, and processes the sentence 'The boys are running, the girls are writing letters, and the children played games.' to generate the POS and Lemma table.

Experiment 4

Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

Python Code

Finite State Machine for Morphological Parsing

Generates plural forms of English nouns

```
def generate_plural(noun):
```

```
    # State q0 (Start State)
```

```
    if noun.endswith(("s", "x", "z", "ch", "sh")):
```

```
        # Transition to q1
```

```
        plural = noun + "es"
```

```
    elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
```

```
        # Transition to q2
```

```
        plural = noun[:-1] + "ies"
```

```
    else:
```

```
        # Transition to q3
```

```
        plural = noun + "s"
```

```
    return plural
```

```
# Test words

words = ["cat", "dog", "bus", "box", "church",
         "dish", "baby", "city", "toy", "book"]

print("Singular\tPlural")

print("-----")

for word in words:

    print(f'{word:10}\t{generate_plural(word)}')
```

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\rishi\AppData\Local\Programs\Python\Python310\nlp lab\nlp4.py
Singular      Plural
-----
cat            cats
dog            dogs
bus            buses
box            boxes
church         churches
dish           dishes
baby           babies
city           cities
toy            toys
book           books
>>>
```

```
# Finite State Machine for Morphological Parsing
# Generates plural forms of English nouns

def generate_plural(noun):
    # State q0 (Start State)
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        # Transition to q1
        plural = noun + "es"
    elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
        # Transition to q2
        plural = noun[:-1] + "ies"
    else:
        # Transition to q3
        plural = noun + "s"
    return plural

# Test words
words = ["cat", "dog", "bus", "box", "church",
         "dish", "baby", "city", "toy", "book"]

print("Singular\tPlural")
print("-----")

for word in words:
    print(f'{word:10}\t{generate_plural(word)}')
```

Experiment 5

Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

Python Code

```
import nltk

from nltk.stem import PorterStemmer

# Create Porter Stemmer object

stemmer = PorterStemmer()

# List of words

words = [

    "running", "playing", "studies", "studying",

    "connected", "connection", "happiness",

    "flying", "wolves", "cars"

]
```

```
print("Original Word\t\tStemmed Word")
```

```
print("-" * 40)
```

```
# Perform stemming
```

```
for word in words:
```

```
    stem = stemmer.stem(word)
```

```
    print(f'{word:20} {stem}')
```



```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/rishi/Downloads/nlp5.py =====
Original Word      Stemmed Word
-----
running            run
playing            play
studies            studi
studying           studi
connected          connect
connection         connect
happiness          happi
flying             fli
wolves             wolv
cars               car
>>>
```

```
File Edit Format Run Options Window Help
nlp5.py - C:/Users/rishi/Downloads/nlp5.py (3.10.11)

import nltk
from nltk.stem import PorterStemmer

# Create Porter Stemmer object
stemmer = PorterStemmer()

# List of words
words = [
    "running", "playing", "studies", "studying",
    "connected", "connection", "happiness",
    "flying", "wolves", "cars"
]

print("Original Word\t\tStemmed Word")
print("-" * 40)

# Perform stemming
for word in words:
    stem = stemmer.stem(word)
    print(f'{word:20} {stem}')
```

EXPERIMENT -6

Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

Morphological Analysis using NLTK (Without Porter Stemmer)

Python code:

Morphological Analysis using NLTK (Without Porter Stemmer)

```
import nltk
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.stem import WordNetLemmatizer
```

```
# Download required resources (Run only once)
```

```
nltk.download('punkt')
```

```
nltk.download('wordnet')
```

```
nltk.download('omw-1.4')
```

```
# Input text
```

```
text = input("Enter a sentence: ")
```

```

# Tokenization

tokens = word_tokenize(text)

# Initialize lemmatizer

lemmatizer = WordNetLemmatizer()

print("\nMorphological Analysis:")

print("-----")

print("{:<15} {}".format("Original", "Lemma"))

print("-----")

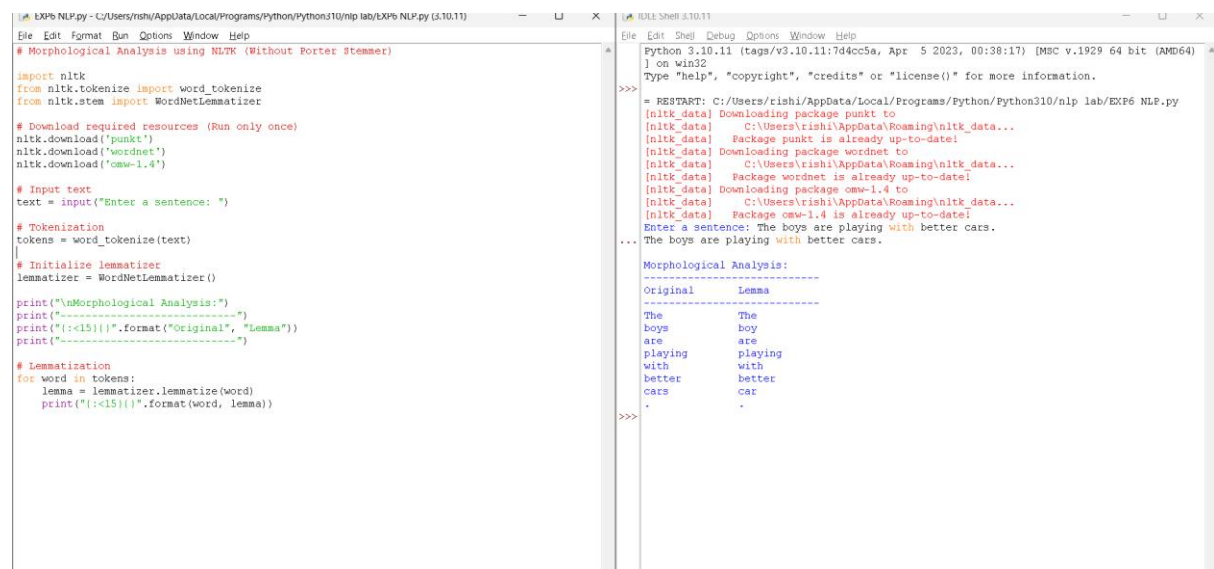
# Lemmatization

for word in tokens:

    lemma = lemmatizer.lemmatize(word)

    print("{:<15} {}".format(word, lemma))

```



The screenshot shows a Python IDE with two panes. The left pane contains the Python code for morphological analysis using NLTK. The right pane shows the output of the script, including the installation of NLTK packages and the results of the lemmatization process.

Code in the left pane:

```

# Morphological Analysis using NLTK (Without Porter Stemmer)

import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Download required resources (Run only once)
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Input text
text = input("Enter a sentence: ")

# Tokenization
tokens = word_tokenize(text)

# Initialize lemmatizer
lemmatizer = WordNetLemmatizer()

print("\nMorphological Analysis:")
print("-----")
print("{:<15} {}".format("Original", "Lemma"))
print("-----")

# Lemmatization
for word in tokens:
    lemma = lemmatizer.lemmatize(word)
    print("{:<15} {}".format(word, lemma))

```

Output in the right pane:

```

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rishi/AppData/Local/Programs/Python/Python310/nlp lab/EXP6 NLP.py
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\rishi\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\rishi\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data] C:\Users\rishi\AppData\Roaming\nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
Enter a sentence: The boys are playing with better cars.
... The boys are playing with better cars.

Morphological Analysis:
-----
Original      Lemma
-----
The           The
boys          boy
are           are
playing       playing
with          with
better        better
cars          car

```